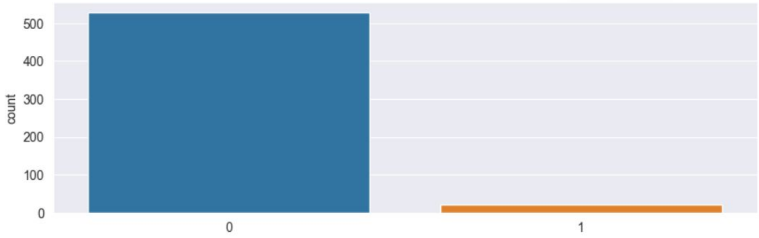
ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE MADRID

Imbalanced Data

Eduardo Fernández Carrión, PhD

El problema



Un conjunto de datos está desbalanceado cuando las clases (o etiquetas) no están representadas de manera igualitaria.

Potenciales impactos:

- Si un modelo se entrena en un conjunto de datos desbalanceado, puede desarrollar un sesgo hacia la clase más representada.
- Las métricas de rendimiento estándar, como el accuracy, pueden no reflejar el verdadero rendimiento del modelo en un conjunto de datos desbalanceado.
- Los algoritmos pueden tener dificultades para aprender patrones de la clase minoritaria debido a la falta de ejemplos.

Casos de uso

- Detección de fraudes financieros
- Diagnóstico médico
- Detección de spam
- Modelos de abandono (churn)
- Modelos de venta cruzada (cross-selling, up-selling)
- Detección de anomalías
- Pronóstico de demanda de productos poco comunes.
- Detección de actividades terroristas en datos de inteligencia.
- Desastres naturales
- Detección de plagios

Desbalanceo agresivo:

- 1 de cada 10
- 1 de cada 100
- 1 de cada 1000
- ...
- 1 de cada 100000
- 1 de cada 1000000

Posibles soluciones

En Python, podemos utilizar la librería de Imbalanced-learn donde se proponen diversos algoritmos que ayudan a entrenar modelos para reducir los problemas descritos.

Potenciales estrategias:

- *Stratification*
- *Random Over Sampling*
- *Random Under Sampling*
- *Smote*
- *Algoritmos preparados para datos desbalanceados*
- *Algoritmos clásicos, pero que permite establecer pesos a las variables en función de la target*

Dataset

- **Embarazos (Pregnancies):** Esta columna representa el número de veces que la persona ha estado embarazada. Un mayor número de embarazos puede estar asociado con un riesgo elevado de desarrollar diabetes tipo 2, especialmente si hubo historial de diabetes gestacional.
- **Glucosa (Glucose):** Indica la concentración de glucosa en plasma en una prueba de tolerancia oral a la glucosa (GTT). Niveles elevados de glucosa en ayunas pueden ser un indicador de diabetes o pre-diabetes.
- **Presión Arterial (BloodPressure):** Muestra la presión arterial diastólica en mm Hg. La hipertensión (alta presión arterial) está a menudo asociada con un mayor riesgo de diabetes tipo 2.
- **Grosor de la Piel (SkinThickness):** Medición del grosor del pliegue de la piel del tríceps en mm. Este parámetro puede ser un indicador de resistencia a la insulina.
- **Insulina (Insulin):** Refleja la concentración de insulina en suero en una unidad de medida (como µU/ml). Niveles anormalmente altos o bajos pueden indicar problemas con la producción de insulina o resistencia a la insulina, ambos asociados con la diabetes.
- **Índice de Masa Corporal (BMI):** Calculado como el peso en kilogramos dividido por el cuadrado de la altura en metros. Un BMI alto puede ser un factor de riesgo para la diabetes.
- **Función de Pedigree de Diabetes (Pedigree):** Un valor que representa la predisposición genética a la diabetes, basado en el historial familiar de diabetes. Es una medida de la probabilidad de diabetes basada en la ascendencia familiar.
- **Edad (Age):** La edad del individuo en años. El riesgo de desarrollar diabetes tipo 2 aumenta con la edad.
- **Diabetes (Outcome o Label):** Esta es generalmente la columna objetivo o de etiqueta en el conjunto de datos. Indica si la persona tiene o no diabetes, a menudo representado como un valor binario (1 para diabetes, 0 para no diabetes).

Load data

```
df = pd.read_csv('https://raw.githubusercontent.com/eduardofc/data/main/diabetes.csv')

print(f'Número de filas: {df.shape[0]}, Número de columnas:{df.shape[1]}')
df.head()
```

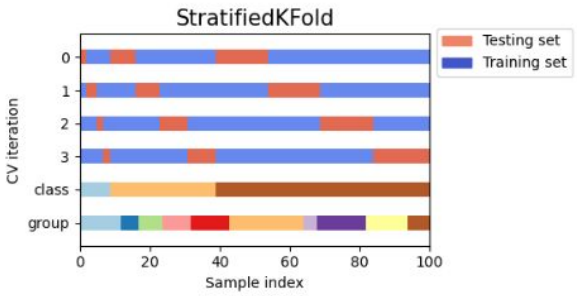
```
Número de filas: 550, Número de columnas:9
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Diabetes
0	10	129	76	28	122	35.9	0.280	39	0
1	4	84	90	23	56	39.5	0.159	25	0
2	0	84	82	31	125	38.2	0.233	23	0
3	9	134	74	33	60	25.9	0.460	81	0
4	0	124	56	13	105	21.8	0.452	21	0

Stratification

Podemos entrenar un modelo con las siguientes características:

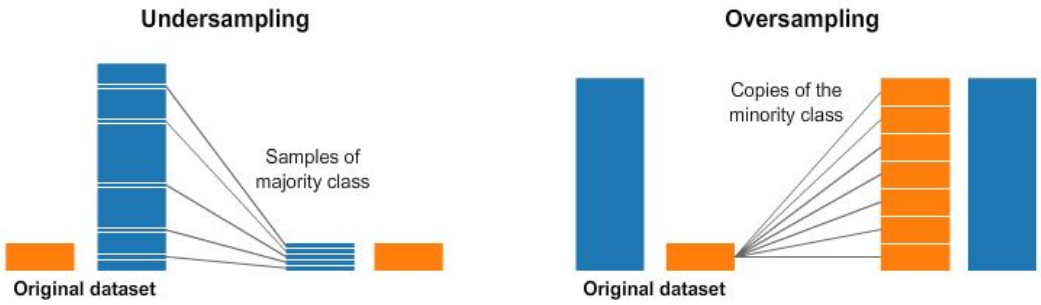
- Train-test-split que contenga el argumento `stratify`.
- CV de tipo `StratifiedKFolds`.
- Métrica prohibida: `accuracy`.
- Métricas recomendadas:
  - `'precision'`,
  - `'recall'`
  - `'f1'`




```
cv_technique = StratifiedKFold(n_splits=5, random_state=seed, shuffle=True)
grid_model = GridSearchCV(estimator=model, cv=cv_technique, scoring="recall", param_grid={})
grid_model = grid_model.fit(X_train, y_train)
y_pred = grid_model.predict(X_test)
```

Over & Undersampling

- Over-sampling implica incrementar el número de instancias en la clase menos representada.
- Under-sampling consiste en reducir el número de instancias de la clase mayoritaria.




```
ros = RandomOverSampler(random_state=seed)
X_resampled, y_resampled = ros.fit_resample(X, y)
```

https://imbalanced-learn.org/stable/under_sampling.html
https://imbalanced-learn.org/stable/over_sampling.html

SMOTE

- SMOTE (Synthetic Minority Over-sampling Technique) es una técnica de **over-sampling**.
- Trata de equilibrar la distribución de clases creando **ejemplos sintéticos**, en lugar de simplemente duplicar los existentes.

1. **Selección de Muestras**: SMOTE selecciona una muestra aleatoria de la clase minoritaria.
2. **Encontrar Vecinos Más Cercanos**: Encuentra los k vecinos más cercanos de esa muestra (típicamente usando una métrica como la distancia euclidiana) dentro de la clase minoritaria.
3. **Crear Muestra Sintética**: Para cada vecino, SMOTE crea una nueva muestra sintética interpolando entre la muestra original y sus vecinos. Esencialmente, esto significa que la nueva muestra es una combinación de las características de la muestra original y su vecino, situándola "entre" ellas en el espacio de características.
4. **Repetir el Proceso**: Este proceso se repite hasta que el número de muestras en la clase minoritaria se equilibra con la clase mayoritaria.

https://imbalanced-learn.org/stable/over_sampling.html#smote-adasyn

Balanced Algorithms

Existen algoritmos diseñados para datasets desbalanceados como **BalancedRandomForestClassifier** y **BalancedBaggingClassifier:**

- los árboles tienden a estar sesgados hacia la clase mayoritaria con un under-sampling

```
from imblearn.ensemble import BalancedRandomForestClassifier

model = BalancedRandomForestClassifier(
    sampling_strategy="all",
    replacement=True,
    max_depth=2,
    random_state=seed,
    bootstrap=False
)
```

https://imbalanced-learn.org/stable/references/generated/imblearn.ensemble.BalancedRandomForestClassifier.html
https://imbalanced-learn.org/stable/references/generated/imblearn.ensemble.BalancedBaggingClassifier.html

Class weight

Algunos algoritmos permiten dar mayor o menor peso a los registros de una clase target específica. Al aumentar el peso de las clases minoritarias, el modelo presta más atención a estas durante el entrenamiento, lo que puede mejorar el rendimiento y la precisión en estas clases.

L**os pesos afectan cómo se calcula la función de pérdida**. Clases con mayor peso tendrán un impacto mayor en la función de pérdida, llevando al modelo a enfocarse en minimizar errores en esas clases.

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None,
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False,
class_weight=None, ccp_alpha=0.0, max_samples=None, monotonic_cst=None)                                    [source]
```

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

ntic master
revolucionamos la comunicación

UNIVERSIDAD COMPLUTENSE MADRID